

Explicit State Space and Markov Chain Generation Using Decision Diagrams

Junaid Babar and Andrew S. Miner*

Department of Computer Science, Iowa State University, Ames IA 50011, USA
{junaid, asminer}@iastate.edu

Abstract. Various forms of decision diagrams have been successfully used for quite some time to generate the state space and Markov chain from models expressed in some high-level formalism. A variety of efficient, “symbolic” algorithms, which manipulate sets of states instead of individual states, are known for this purpose. However, there are cases where explicit generation algorithms are still used. This paper seeks to efficiently use decision diagrams as replacement data structures within an existing explicit generation implementation. The necessary decision diagram algorithms are presented, and small changes to the explicit generation algorithm are suggested to improve the overall generation process. The efficiency of the new algorithms is illustrated using several models.

Keywords: Quantitative model checking, Markov chain generation, Markov chain storage, decision diagrams

1 Introduction

Generating the state space and underlying Markov chain from a model described in a high-level formalism, such as a Petri net, is a necessary first step for many types of analysis. Important applications that utilize this information include performance evaluation (e.g., [22, 23]) and stochastic model checking (e.g., [3]). The well-known *state explosion problem*, in which a “small” high-level model may describe a huge state space and Markov chain, effectively limits the size and complexity of systems that may be analyzed in this manner.

A successful approach to help manage the state explosion problem has been the use of so-called “symbolic” or implicit algorithms, which utilize *decision diagrams* (DDs) (e.g., [4, 16]), a structure that represents finite sets of integer vectors compactly for many (but not all) practical sets. As the symbolic algorithms work directly with DDs, by building a transition relation from the high level model and constructing the state space by a sequence of operations on DDs, they are limited by the sizes of the DDs generated, which can remain small even for huge state spaces. Once this was realized [5], many other researchers adopted the idea and expanded upon it; for example, by applying it to Petri nets [24], by developing generation algorithms [9], or by proposing variants of DDs [12, 17, 20,

* This work was supported by the National Science Foundation, grant CNS-0546041.

30]. In particular, several variants [15, 19, 26] have been proposed for representing Markov chains by including the rate information in the transitions relations.

While symbolic generation algorithms have enjoyed widespread success, they have not *completely* replaced explicit algorithms. For Markov chain analysis in particular, there are cases where explicit algorithms are often still used in practice. Notably, algorithms that exploit symmetry, such as the generation algorithms for Stochastic Well-Formed Nets [8] that immediately build a lumped process, remain explicit, despite preliminary work to make them symbolic [13]. For models containing immediate events, it is non-trivial to symbolically construct a Markov chain over the “tangible” states (e.g., [20]). Finally, even when DDs compactly represent large Markov chains, since the numerical solution of the Markov chain usually requires one or more *explicit* solution vectors [18], in practice, explicit generation algorithms are often considered “good enough”.

In this paper, we propose the use of DDs as data structures to represent the state space and underlying Markov chain while using *explicit* generation algorithms. This approach is not intended to compete with symbolic generation algorithms (although, for certain models, symbolic generation can be slow and explicit generation is a viable alternative); rather, the intention is to allow cases that cannot (or for various reasons, do not) use symbolic generation, such as those discussed above, to utilize DDs. As DDs are usually much more compact than traditional explicit data structures, the goal is to reduce the memory requirements during generation, and to do so without greatly increasing the generation times. Once the state space and underlying Markov chain have been generated as DDs, the numerical solution algorithms in [18], using explicit solution vectors and the DD representation of the Markov chain, may be invoked.

In addition to numerical solution, there are other instances when explicit algorithms are used with DDs. Algorithms for classifying the states of a Markov chain, stored using DDs, into recurrent and transient classes, are partly explicit: the initial “seed” states are chosen, one at a time, at random; each seed state and potentially a large set of other states may then be classified using symbolic operations [29]. The “symblicit” approach for analyzing Markov Decision Processes presented in [27] is a combination of explicit and symbolic approaches. Similarly, the lumping algorithm in [14] contains explicit loops over states.

However, for state space and Markov chain or reachability graph generation, most algorithms are either completely explicit or completely symbolic. We are only aware of a few exceptions to this rule, and these previous works are most relevant to this paper. In [19], a type of DD called *matrix diagram* is used to store the Markov chain, and an explicit algorithm is used to generate the Markov chain. However, this work assumes that the state space is already known and is represented using a DD. Recently, in [1], an explicit state space generation algorithm was integrated into GreatSPN [6]; however, this uses a fairly simple algorithm. This paper builds upon these works, with extensions and generalizations over [19], and algorithm improvements over [1].

The remainder of this paper is organized as follows. Section 2 briefly recalls the traditional, explicit generation algorithms and motivates the data structure

requirements. Section 3 formally defines DDs and some variants, and reviews some necessary, known algorithms. Section 4 describes how DDs can be efficiently integrated into explicit generation algorithms, and discusses some improvements to make this more efficient. Section 5 gives experimental results for our new algorithms, and section 6 concludes the work.

2 Background

Instead of a specific high-level formalism, we use an abstract model definition, and treat the model as a “black box”. We assume that a model consists of:

- A set of L state variables, $x_L, \dots, x_1$. Each state variable may assume a finite number of possible values; for simplicity, we assume these are the first naturals, i.e., $x_l \in \mathcal{S}_l = \{0, 1, \dots, n_l - 1\}$. The possible states of the model are therefore $\hat{\mathcal{S}} = \mathcal{S}_L \times \dots \times \mathcal{S}_1$; a finite set. While we assume the existence of bounds $n_L, \dots, n_1$, we do not require that these are known a priori.
- An initial distribution, $\pi_0 : \hat{\mathcal{S}} \rightarrow [0, 1]$, with the restriction $\sum_{\mathbf{s} \in \hat{\mathcal{S}}} \pi_0(\mathbf{s}) = 1$.
- A next-state function, $\mathcal{N} : \hat{\mathcal{S}} \rightarrow 2^{\hat{\mathcal{S}} \times \mathbb{R}^+}$, where $\mathbb{R}^+$ denotes the set of positive reals. The meaning of $(\mathbf{s}', r) \in \mathcal{N}(\mathbf{s})$ is that the model can change from state $\mathbf{s}$ to state $\mathbf{s}'$ in one step, with rate r .

The state space of the model, $\mathcal{S}$, is the set of all states that can be reached from the initial states, when zero or more actions occur. Formally, the set of initial states may be defined as $\mathcal{I} = \{\mathbf{s} : \pi_0(\mathbf{s}) > 0\}$, and $\mathcal{S}$ is the smallest superset of $\mathcal{I}$ that satisfies $\mathbf{s} \in \mathcal{S} \Rightarrow \forall (\mathbf{s}', r) \in \mathcal{N}(\mathbf{s}), \mathbf{s}' \in \mathcal{S}$. The (reachable) Markov chain of the model, $\mathbf{R} : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}^{\geq 0}$, where $\mathbb{R}^{\geq 0}$ denotes the set of non-negative reals, is defined as $\mathbf{R}(\mathbf{s}, \mathbf{s}') = \sum_{(\mathbf{s}', r) \in \mathcal{N}(\mathbf{s})} r$.

A traditional algorithm to generate the state space and Markov chain from a model is shown in Figure 1(a). Sometimes, it may be useful to generate the Markov chain after the state space is known (for example, by collecting information while generating the state space, a more compact data structure may be used for the Markov chain); this may be done using the two-pass algorithm shown in Figure 1(b). Both algorithms require data structures for $\mathcal{S}$, the currently-known state space; $\mathcal{U}$, a set of states that still need to be explored; and $\mathbf{R}$, the currently-known Markov chain. Note that each algorithm terminates if and only if the state space is finite. After termination, $\mathcal{S}$ and $\mathbf{R}$ will respectively hold the complete state space and Markov chain, and $\mathcal{U}$ will be empty.

From a data structure perspective, the critical operations for $\mathcal{S}$ are adding states (cf. line 8) and determining if a given state is contained in the set (cf. line 7). The critical operations for $\mathcal{U}$ are checking if the set is empty (cf. line 4), adding states (cf. line 9), and removing *some* state (cf. line 5). Note that if $\mathcal{U}$ removes states in FIFO order (i.e., $\mathcal{U}$ is a queue), then the generation algorithm uses breadth-first search; however, this is not required. A critical operation applicable only to the two-pass algorithm is enumeration (cf. line 13). The only critical operation for $\mathbf{R}$ is to add edges; note that this operation should allow for the destination state to appear more than once, i.e., we could have a model with $(\mathbf{s}', r_1), (\mathbf{s}', r_2) \in \mathcal{N}(\mathbf{s})$.

<pre> 1: $\mathcal{S} \leftarrow \{s : \pi_0[s] > 0\};$ 2: $\mathcal{U} \leftarrow \mathcal{S};$ 3: $\mathbf{R} \leftarrow \mathbf{0};$ 4: while $\mathcal{U} \neq \emptyset$ do 5: Remove some s from $\mathcal{U};$ 6: for all $(s', r) \in \mathcal{N}(s)$ do 7: if $s' \notin \mathcal{S}$ then 8: $\mathcal{S} \leftarrow \mathcal{S} \cup \{s'\};$ 9: $\mathcal{U} \leftarrow \mathcal{U} \cup \{s'\};$ 10: end if 11: $\mathbf{R}(s, s') \leftarrow \mathbf{R}(s, s') + r;$ 12: end for 13: end while </pre>	<pre> 1: $\mathcal{S} \leftarrow \{s : \pi_0[s] > 0\};$ 2: $\mathcal{U} \leftarrow \mathcal{S};$ 3: $\mathbf{R} \leftarrow \mathbf{0};$ 4: while $\mathcal{U} \neq \emptyset$ do 5: Remove some s from $\mathcal{U};$ 6: for all $(s', r) \in \mathcal{N}(s)$ do 7: if $s' \notin \mathcal{S}$ then 8: $\mathcal{S} \leftarrow \mathcal{S} \cup \{s'\};$ 9: $\mathcal{U} \leftarrow \mathcal{U} \cup \{s'\};$ 10: end if 11: end for 12: end while 13: for all $s \in \mathcal{S}$ do 14: for all $(s', r) \in \mathcal{N}(s)$ do 15: $\mathbf{R}(s, s') \leftarrow \mathbf{R}(s, s') + r;$ 16: end for 17: end for </pre>
(a) One-pass algorithm	(b) Two-pass algorithm

Fig. 1. Algorithms to generate the state space and Markov chain from a model

3 Decision Diagrams

There are many possible data-structures that can be used for storing $\mathcal{S}$, $\mathcal{U}$, and $\mathbf{R}$, and the design choice for one usually affects the others. A traditional strategy for the generation of $\mathcal{S}$ and $\mathbf{R}$, is:

- store an explicit (but compressed) representation of the states so that each state has a unique index that coincides with the discovery order of the states;
- use a classical dictionary data structure (such as a splay tree or hash table) to determine if states are already contained in $\mathcal{S}$;
- use an integer u for $\mathcal{U}$, to specify that all states with an index greater than or equal to u are yet to be explored;
- and use a dynamic data structure for directed, weighted, sparse graphs, such as adjacency lists, for $\mathbf{R}$.

See, for example, [7] for a survey of efficient techniques for explicit generation.

This paper proposes the use of Decision Diagrams (DDs) instead of the explicit structures mentioned above to store $\mathcal{S}$, $\mathcal{U}$ and $\mathbf{R}$. A DD is a directed acyclic graph used to represent a function on a finite number of variables, where each variable can assume a finite number of values. To simplify the discussion, we assume that the DD variables exactly match the state variables of the model. However, this is not required; it is possible (and often desired) to split a single model variable into several DD variables (e.g., [15]), or conversely to group several model variables into a single DD variable (e.g., [9]).

Each DD node is either *terminal*, with no outgoing edges, or *non-terminal*, labeled with a variable x_l and containing n_l outgoing edges to other nodes. For a given non-terminal node p , we use $p[i]$ to denote the i^{th} outgoing edge. Ordered DDs require a *total ordering* $\succ$ on the variables such that any outgoing edge from a node labeled x_l must go to either a terminal node or to a node labeled x_k with $x_l \succ x_k$. In this paper, we assume that all DDs are Ordered.

DDs employ *reduction rules* to eliminate duplicate nodes, and to determine cases in which nodes are eliminated; done carefully, this ensures that each DD variant is a *canonical* representation. Two non-terminal nodes p and q are *duplicates* if they have the same variable label x_l , and the same outgoing edges: $p[i] = q[i], \forall 0 \leq i < n_l$. All DDs in this paper eliminate duplicate nodes. Additional reduction rules, combined with different variable types and ranges, produce a variety of DDs:

- Multiway Decision Diagrams (MDDs) are used to represent functions of the form $f : \hat{\mathcal{S}} \rightarrow \{0, 1\}$, and are typically *fully-reduced*, eliminating the pattern of a variable that does not matter. If each variable is boolean ($n_l = 2, \forall l$) then these become classical BDDs [4]. An MDD can represent a set of model states, by encoding the characteristic function of the set.
- Matrix Diagrams (MxDs) are used to represent functions of the form $f : \hat{\mathcal{S}} \times \hat{\mathcal{S}} \rightarrow \{0, 1\}$, using an “interleaved” ordering $x_L \succ x'_L \succ \dots \succ x_1 \succ x'_1$. These are typically *identity-reduced*, eliminating the pattern of a variable that does not change (e.g., $x_l = x'_l$) [20].
- Multi-terminal Matrix Diagrams (MTMxDs) are used to represent functions of the form $f : \hat{\mathcal{S}} \times \hat{\mathcal{S}} \rightarrow \mathbb{R}$, and are essentially the same as MxDs except the terminal nodes are labeled with real values.
- Multiplicative Edge-Valued Matrix Diagrams (EV*MxDs) also represent functions of the form $f : \hat{\mathcal{S}} \times \hat{\mathcal{S}} \rightarrow \mathbb{R}$, but store real values along each edge in the DD, and values are multiplied along the path from the root to the terminal node to obtain the encoded value. A *normalization rule* (e.g., “the first non-zero edge value must be 1.0”) is necessary to preserve canonicity. EV*MxDs are essentially EV*MDDs [26] that utilize the notions of interleaved variable ordering and identity reductions from MxDs.
- Canonical Matrix Diagrams (CMDs) [19] are an earlier version of EV*MxDs, where each node is a *matrix* of outgoing edges; effectively, a node for variable x_l in a CMD corresponds (roughly) to nodes for variables x_l and x'_l in an EV*MxD. Note that the CMDs of [19] did *not* utilize the identity reduction.

As a simple example, consider an open network of 3 bounded queues, where arrivals enter queue 3, queue 3 feeds into queue 2, queue 2 feeds into queue 1, and departures from queue 1 exit the system. If each queue can contain at most two customers, then the model has $\mathcal{S}_3 = \mathcal{S}_2 = \mathcal{S}_1 = \{0, 1, 2\}$. Figure 2 shows the MTMxD, EV*MxD, and CMD representations for the underlying Markov chain for this system, using an arrival rate of 3.0, and service rates of 4.0, 3.2, and 3.6 for queues 3, 2, and 1, respectively. In the illustrations, edges to terminal node 0 are omitted for clarity. For the CMD illustration, edges with the same edge value

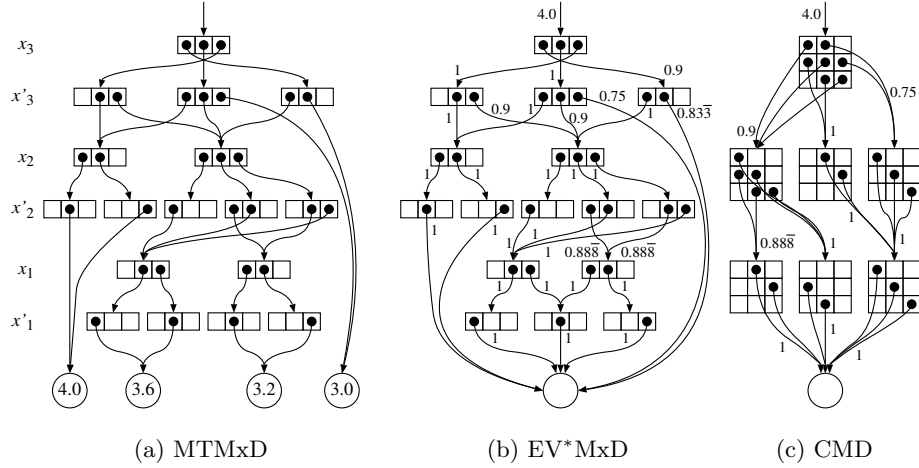


Fig. 2. Comparison of DDs encoding the same Markov chain

are clustered together, with only the common edge value drawn. EV*MxD and CMD use the normalization rule: “the largest edge value out of a node is 1.0”.

There are a variety of operations that may be performed on DDs. Typically these operations traverse the DDs recursively, and utilize a compute table to avoid duplication of work since the recursion could visit the same node several times. For the purpose of this work, we note that for MDDs encoding sets of states, there are efficient operations to take the union or difference of two sets, determine the cardinality of a set, or enumerate the elements of a set. For an EV*MxD, MTMxD, or CMD representation of a Markov chain, it is possible to compute measures of interest (e.g., the steady-state distribution) using an *explicit* solution vector of size $|\mathcal{S}|$. For more details, please refer to [18].

4 Explicit Generation with Decision Diagrams

Our goal is to use either the one-pass or two-pass algorithm shown in Figure 1, with MDDs for $\mathcal{S}$ and $\mathcal{U}$, and EV*MxDs, MTMxDs, or CMDs for $\mathbf{R}$. To do so, we must address the critical operations discussed in Section 2, namely: adding elements, removing elements, checking if $\mathbf{s} \in \mathcal{S}$, and checking if $\mathcal{U} \neq \emptyset$.

Because any given DD node can have several incoming edges, most DD algorithms do not modify DDs in place, but rather construct entirely new DDs. Thus, a statement such as “ $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{s}'\}$ ” would require constructing an MDD encoding the set $\{\mathbf{s}'\}$ (which can be done using procedure `state2MDD` shown in Figure 3), and then invoking a union operation on two MDDs. Since the union operation can be rather heavy, we instead use a fixed-size *state buffer* to collect states, so that we can reduce the number of union operations. Thus, we need an algorithm to efficiently build an MDD encoding a set $\{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_b\}$ where each state $\mathbf{s}_i$ is stored explicitly in an array. This can be done using procedure

<pre> node states2MDD(var L, state buffer $\mathbf{b}$) 1: if $L = 0$ then 2: return terminal node 1; 3: end if 4: if $\mathbf{b}$ has only one state then 5: return state2MDD($L, \mathbf{b}[0]$); 6: end if 7: RadixSort $\mathbf{b}$ using variable x_L; 8: $p \leftarrow$ new node at level L; 9: for all values v for x_L in $\mathbf{b}$ do 10: $\mathbf{b}' \leftarrow$ states in $\mathbf{b}$ with $x_L = v$; 11: $p[v] \leftarrow$ states2MDD($L - 1, \mathbf{b}'$); 12: end for 13: return Reduce(p); </pre>	<pre> node state2MDD(var L, state $\mathbf{s}$) 1: $p \leftarrow$ terminal node 1; 2: for all $l \in \{1, \dots, L\}$ do 3: $q \leftarrow$ new node at level l 4: $q[\mathbf{s}[l]] \leftarrow p$; 5: $p \leftarrow$ Reduce(q); 6: end for 7: return p; </pre>
--	---

Fig. 3. Algorithm to build an MDD from an array of states

“states2MDD” shown in Figure 3. The procedure is recursive by variable number. The state array is sorted by the current variable (c.f. line 7), and the node at the current level is constructed by building the children recursively based on the sub-arrays grouped by variable value (c.f. lines 10 and 11). Note that these states are contiguous because of the sort, so in practice $\mathbf{b}'$ can be built from $\mathbf{b}$ inexpensively. Recursion continues until either the bottom level is reached, or the array of states shrinks down to a single state. It can be shown that, because the states are sorted, MDD nodes are created bottom-up without creating any intermediate nodes. In other words, every node that is created, will appear in the “final” MDD, unless it is eliminated by the reduction step (c.f., line 13). The algorithm can be modified as appropriate to add matrix elements to an $\text{EV}^* \text{MxD}$ or MTMxD ; we call that procedure “rates2MxD”.

One way to implement the statement “Remove some $\mathbf{s}$ from $\mathcal{U}$ ”, is to find a state $\mathbf{s} \in \mathcal{U}$ by modifying the algorithm to enumerate elements of an MDD (stopping after the first element), build an MDD for the set $\{\mathbf{s}\}$ using “state2MDD”, and invoke the set difference operation. Taking this approach, and combining it with the “batch addition” idea discussed above, we can modify the one-pass algorithm to obtain the “Single Removal” algorithm shown in Figure 4(a). “ $\mathcal{U} \neq \emptyset$ ” can be determined easily due to the fact that MDDs are a canonical representation, and “ $\mathbf{s} \in \mathcal{S}$ ” can be determined by following the appropriate path through the MDD for $\mathcal{S}$, until a terminal node is reached, 0 for $\mathbf{s} \notin \mathcal{S}$, and 1 for $\mathbf{s} \in \mathcal{S}$. Note that the state buffer, $\mathbf{b}_S$, must be flushed whenever the set $\mathcal{U}$ becomes empty (c.f. lines 22–24), otherwise the algorithm may terminate prematurely. Similarly, the rate buffer $\mathbf{b}_R$ must be flushed before terminating (c.f. line 27).

Performing a set difference operation for each element of $\mathcal{U}$ (i.e. every discovered state) could be expensive. We might save time by removing states in batches. As described earlier, the single-removal algorithm removes a state $\mathbf{s}$ from $\mathcal{U}$ and uses it to discover new states. We need a procedure to obtain $\mathbf{s}$ from $\mathcal{U}$ without performing a set difference operation on $\mathcal{U}$, while permitting $\mathcal{U}$ to be

<pre> 1: $\mathcal{S} \leftarrow \{s : \pi_0[s] > 0\}$; 2: $\mathcal{U} \leftarrow \mathcal{S}$; $\mathbf{b}_S \leftarrow \emptyset$; $\mathbf{b}_R \leftarrow \mathbf{0}$; 3: while $\mathcal{U} \neq \emptyset$ do 4: $s \leftarrow \text{FindFirstElement}(\mathcal{U})$; 5: $\mathcal{U} \leftarrow \mathcal{U} \setminus \text{state2MDD}(L, s)$; 6: for all $(s', r) \in \mathcal{N}(s)$ do 7: if $s' \notin \mathcal{S}$ then 8: Add s' to $\mathbf{b}_S$; 9: if $\mathbf{b}_S$ is full then 10: $\mathcal{S}' \leftarrow \text{states2MDD}(L, \mathbf{b}_S)$; 11: $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}'$; $\mathcal{U} \leftarrow \mathcal{U} \cup \mathcal{S}'$; 12: $\mathbf{b}_S \leftarrow \emptyset$; 13: end if 14: end if 15: Add (s, s', r) to $\mathbf{b}_R$; 16: if $\mathbf{b}_R$ is full then 17: $\mathbf{R} \leftarrow \mathbf{R} + \text{rates2MxD}(L, \mathbf{b}_R)$; 18: $\mathbf{b}_R \leftarrow \mathbf{0}$; 19: end if 20: end for 21: if $\mathcal{U} = \emptyset$ then 22: $\mathcal{S}' \leftarrow \text{states2MDD}(L, \mathbf{b}_S)$; 23: $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}'$; $\mathcal{U} \leftarrow \mathcal{U} \cup \mathcal{S}'$; 24: $\mathbf{b}_S \leftarrow \emptyset$; 25: end if 26: end while 27: $\mathbf{R} \leftarrow \mathbf{R} + \text{rates2MxD}(L, \mathbf{b}_R)$; </pre>	<pre> 1: $\mathcal{S} \leftarrow \{s : \pi_0[s] > 0\}$; 2: $\mathcal{U} \leftarrow \mathcal{S}$; $\mathbf{b}_S \leftarrow \emptyset$; $\mathbf{b}_R \leftarrow \mathbf{0}$; 3: while $\mathcal{U} \neq \emptyset$ do 4: $\mathcal{U}' \leftarrow \mathcal{U}$; $\mathcal{U} \leftarrow \emptyset$; 5: for all $s \in \mathcal{U}'$ do 6: for all $(s', r) \in \mathcal{N}(s)$ do 7: if $s' \notin \mathcal{S}$ then 8: Add s' to $\mathbf{b}_S$; 9: if $\mathbf{b}_S$ is full then 10: $\mathcal{S}' \leftarrow \text{states2MDD}(L, \mathbf{b}_S)$; 11: $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}'$; $\mathcal{U} \leftarrow \mathcal{U} \cup \mathcal{S}'$; 12: $\mathbf{b}_S \leftarrow \emptyset$; 13: end if 14: end if 15: Add (s, s', r) to $\mathbf{b}_R$; 16: if $\mathbf{b}_R$ is full then 17: $\mathbf{R} \leftarrow \mathbf{R} + \text{rates2MxD}(L, \mathbf{b}_R)$; 18: $\mathbf{b}_R \leftarrow \mathbf{0}$; 19: end if 20: end for 21: end for 22: $\mathcal{S}' \leftarrow \text{states2MDD}(L, \mathbf{b}_S)$; 23: $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}'$; $\mathcal{U} \leftarrow \mathcal{U} \cup \mathcal{S}'$; 24: $\mathbf{b}_S \leftarrow \emptyset$; 25: end while 26: $\mathbf{R} \leftarrow \mathbf{R} + \text{rates2MxD}(L, \mathbf{b}_R)$; </pre>
(a) Single Removal	(b) Batch Removal

Fig. 4. One-pass algorithm, modified to use batch addition

updated within the loop (Figure 4(a), line 11). Our solution is to copy $\mathcal{U}$ into *another* MDD $\mathcal{U}'$ and then enumerate the elements in $\mathcal{U}'$. This gives us all the elements in $\mathcal{U}$ (via $\mathcal{U}'$), eliminates the set difference operation, and allows $\mathcal{U}$ to be modified as needed. Note that copying an MDD as described above is a trivial operation. The “Batch Removal” algorithm is shown in Figure 4(b).

We now make several observations about these algorithms. First, note that the choice of removing the “first element” from $\mathcal{U}$ in the single-removal algorithm is arbitrary; any element will work. The order of the traversal of $\mathcal{U}'$ in the batch-removal algorithm is also arbitrary, but efficient algorithms exist [18] for lexicographical or reverse lexicographical order. Changing these could affect the order in which states are explored. Second, the order in which states are discovered and added to the batches could affect both the “intermediate” MDDs representing $\mathcal{S}$ (it does not affect the final MDD for $\mathcal{S}$ due to MDD canonicity), and the overall computational cost, since the union operations will be performed on different sets. Thus, while the procedure “states2MDD” is optimal for build-

Model	$ \mathcal{S} $	$ \mathbf{R} $	Model	$ \mathcal{S} $	$ \mathbf{R} $
Courier 3	999,450	5,544,240	Kanban 5	2,546,432	24,460,016
Courier 4	5,133,600	31,392,540	Kanban 6	11,261,376	115,708,992
FMS 8	4,459,455	42,302,007	Kanban 7	41,644,800	450,455,040
FMS 9	11,058,190	108,084,295	QN 60	11,510,083	74,544,210
FMS 10	25,397,658	234,523,289	QN 80	58,214,137	384,196,600

Table 1. Markov chain sizes for the models

ing an MDD from a set of states, the same *cannot* be said for the single-removal and batch-removal algorithms. More precisely: there is at least one order of state exploration that will be optimal in terms of size of the largest intermediate MDD, and it is unknown whether either of the algorithms in Figure 4 uses such an order. Third, note that it is possible to add the same unexplored state to the batch $\mathbf{b}_S$ more than once; this happens when a state is discovered again before the batch is added to $\mathcal{S}$. Since each reachable state is guaranteed to be explored exactly once, the overhead of duplication is somewhat contained, and we suspect that this overhead is much smaller than the overhead of a mechanism to detect duplicate states in a batch. Finally, Figure 4 gives the modifications for single removal and batch removal for the one-pass algorithm only; the analogous two-pass algorithms easily follow from this, and are omitted for space.

5 Experimental Results

The DD algorithms discussed in Section 4 are implemented in Meddly [2], and our explicit generation algorithms based on DDs are implemented in SMART [10]. Experiments are based on version 3.2.1411 of SMART and version 0.12.526 of Meddly, and were run on a typical desktop machine: a 2.7 GHz Intel Core i5 processor with 8 Gb of 1333 MHz DDR3 memory, running Mac OS 10.8.5.

5.1 Models

We tested our algorithms on a variety of models, mostly taken from the literature, described below. Table 1 shows, for each model, the size of the Markov chain described by the model, where $|\mathcal{S}|$ denotes the number of states in the Markov chain, and $|\mathbf{R}|$ denotes the number of non-zero entries in the rate matrix.

The Courier model [28] describes a communications protocol, where model parameter N corresponds to the window size and transport space parameters in [28] (i.e., we use $M = N$). This Petri Net contains 45 places, and we use a single place for each MDD variable. Some places may contain up to N tokens, while other places may contain only up to 1 token.

The FMS model [11] describes a flexible manufacturing system where model parameter N specifies the number of pallets to move parts. The model is partitioned into 19 MDD variables, where the number of possible values for most of

the state variables grows with N , while the rest are fixed. The Petri Net for this system utilizes marking-dependent arc cardinalities, marking-dependent firing rates, and immediate transitions.

The Kanban model [25] describes a manufacturing system. The model parameter, N , determines the number of parts circulating in the system. This Petri net contains 16 places, and we assign each place to its own MDD variable.

QN is a classless, closed queueing network model. Initially, N customers are in an infinite-server queue, “pool”. From this queue, customers arrive into a single-server “dispatcher” queue, which quickly sends customers to a “back end” queue with shortest queue length. Each of the 5 single-server “back end” queues process customers, and send them back into the “pool”. All queues have exponentially-distributed service, with different rates, and process customers on a first come, first served policy. We use a decomposition of 7 MDD variables, one for each queue.

We also present some preliminary results obtained by incorporating the one-pass algorithm into GreatSPN [6]. The following colored nets were used: (a) CSRepetitions [21]: a client/server application where communications from clients to servers is unreliable and requests are buffered, (b) DrinkVendingMachine [21]: a symmetric net modeling a hot drink vending machine, and (c) GlobalResAllocation [21]: a resource management model that prevents deadlocks.

5.2 Results

Table 2 shows the generation times and memory required to generate the state space $\mathcal{S}$ and Markov chain $\mathbf{R}$ for each model, using the one-pass algorithm, which constructs $\mathcal{S}$ and $\mathbf{R}$ simultaneously. Table 3 shows similar information for the two-pass algorithm, which constructs $\mathcal{S}$ first, and then constructs $\mathbf{R}$. Generation times are based on CPU usage and are reported in seconds. Memory requirements are based on “peak” sizes of the data structure, and are reported in appropriate units (“k” for 1024 bytes, and “m” for 1024² bytes).

In both tables, “BST” uses a balanced binary search tree (splay tree) for $\mathcal{S}$, and an explicit sparse matrix representation for $\mathbf{R}$; both are typical choices in practice. “EV*” uses an MDD for $\mathcal{S}$, and an EV*MxD for $\mathbf{R}$, as implemented in Meddly. “MT” uses an MDD for $\mathcal{S}$, and an MTMxD for $\mathbf{R}$, as implemented in Meddly. “CMD” uses an MDD for $\mathcal{S}$ as implemented in Meddly, and a Canonical Matrix Diagram for $\mathbf{R}$, using the implementation from [19] with some changes (discussed below). Finally, “br” stands for “batch removal”, and “sr” stands for “single removal”. We use a batch size of 1024 for all experiments.

In Table 2, the CMD implementation was modified to construct CMD nodes from a batch of entries, namely, an array of source states, destination states, and rates. This modification allows the use of CMDs without knowledge of $\mathcal{S}$, as required in [19]. However, as the CMD implementation is rather specialized, it requires a known bound for each MDD variable a priori. In contrast, the EV*MxD and MTMxD implementations in Meddly are designed to be more robust and general-purpose; these instead allow the MDD variable bounds to expand as needed. Looking at the generation times, we see that “BST” is always

Model	Generation times (seconds)						Peak memory required (k=1024 bytes, m=1024k)																			
	BST		EV*		MT		CMD		BST			br EV*			sr EV*			br MT			br CMD			sr CMD		
	br	sr	br	sr	br	sr	br	sr	S	R	S	R	S	R	S	R	S	R	S	R	S	R	S	R		
Courier 3	10	34	54	47	56	39	56	36m	114m	408k	2.2m	2.2m	626k	422k	4.0m	2.1m	943k	588k	3.4m	463k	1.8m					
Courier 4	57	205	284	277	300	244	313	194m	638m	662k	13m	2.2m	738k	664k	5.8m	2.0m	1017k	1.1m	6.2m	473k	2.1m					
FMS 8	72	116	157	211	183	141	1653	157m	769m	326k	2.8m	542k	619k	378k	5.5m	1.9m	1.4m	441k	14m	116k	7.5m					
FMS 9	193	303	397	603	497	379	425	390m	1977m	452k	4.4m	547k	851k	569k	11m	3.8m	2.5m	538k	28m	120k	16m					
FMS 10	—	731	939	1540	1224	954	999	—	—	574k	5.9m	554k	1.1m	693k	14m	7.6m	4.3m	817k	54m	121k	31m					
Kanban 5	22	35	60	50	66	45	64	80m	486m	120k	1.3m	332k	211k	131k	1.2m	1005k	376k	170k	1.4m	151k	845k					
Kanban 6	110	165	275	244	315	217	290	358m	2294m	179k	5.1m	346k	253k	196k	1.5m	1019k	442k	302k	2.0m	163k	1014k					
Kanban 7	—	650	1052	1006	1229	866	1104	—	—	260k	17m	362k	291k	309k	2.6m	1.0m	499k	374k	2.9m	178k	1.2m					
QN 60	56	102	207	673	473	170	251	356m	1506m	209k	1.9m	595k	517k	301k	3.8m	932k	898k	268k	8.4m	151k	3.1m					
QN 80	—	531	1175	4425	3327	1031	1463	—	—	401k	5.9m	643k	902k	628k	8.5m	1.9m	1.7m	533k	20m	195k	4.6m					

Table 2. CPU and peak memory requirements using the **one-pass algorithm** for state space and Markov chain generation

Model	Generation of $\mathcal{S}$						Generation of $\mathbf{R}$											
	Time (seconds)			Peak memory			Time (seconds)			CMD			Peak memory			CMD		
	BST	MDD	br	sr	BST	MDD	BST	EV*	MT	bat.	f = 3	f = 8	BST	EV*	MT	bat.	f = 3	f = 8
Courier 3	9	29	109	36m	947k	508k	10	13	15	17	38	18	114m	327k	352k	767k	186k	197k
Courier 4	54	165	595	194m	1.2m	518k	64	76	85	95	218	104	638m	378k	674k	1.6m	290k	303k
FMS 8	66	133	404	157m	428k	126k	88	62	82	68	64	71	769m	712k	1.6m	7.5m	1.5m	4.4m
FMS 9	173	344	1069	390m	611k	131k	239	160	225	176	168	194	1977m	1.1m	3.8m	16m	2.1m	6.6m
FMS 10	—	840	2585	—	745k	133k	—	378	557	424	404	485	—	1.3m	5.6m	30m	2.8m	9.8m
Kanban 5	20	10	44	80m	170k	151k	27	20	26	21	24	23	486m	100k	172k	311k	145k	2.9m
Kanban 6	101	48	202	358m	302k	163k	166	92	126	106	116	127	2294m	115k	191k	351k	198k	5.9m
Kanban 7	—	187	763	—	374k	178k	—	359	521	391	419	648	—	133k	215k	397k	262k	12m
QN 60	47	37	166	356m	268k	151k	73	60	266	102	102	183	1506m	519k	1.2m	2.5m	1.9m	51m
QN 80	—	188	957	—	533k	195k	—	310	2085	587	581	1001	—	948k	2.3m	4.2m	3.8m	66m

Table 3. CPU and peak memory requirements using the **two-pass algorithm** for state space and Markov chain generation

Model	No Batches		One-Pass EV*				WNRG
	Time	Memory	Time	Memory	Time	Memory	
CSRepetitions	13	10.92	4	2.09	1	1.40	3
DrinkVendingMachine	28	77.22	5	5.97	2	5.45	4
GlobalResAllocation	144	109.07	38	14.49	9	4.86	8

Table 4. Generation using GreatSPN (time in seconds, peak memory in MB)

the fastest, followed by “EV*” and then “CMD” using batch removal, which are quite close except for large models. Batch removal tends to be faster than single removal, except for some models when MTMxDs are used. Recall from the discussion in Section 4, that the order in which states are explored can affect the computational cost of the batch additions in the DDs; it must be the case for these models that single removal happens to result in a significantly better order for MTMxDs, enough to offset the performance improvement of removing states in batch. Looking instead at memory usage, we see that “BST” always uses orders of magnitude more memory than “EV*”, “MT”, or “CMD”. In particular, for several models, “BST” required more memory than was available on the machine, and could not complete; these cases are marked with the entries “—”. Also, we note that, with some exceptions, “EV*” tends to have the smallest memory requirement, followed by “MT” and then “CMD”. However, assuming the ultimate goal is to analyze the Markov chain numerically, it is perhaps unnecessary to compare *peak* memory usage between “EV*”, “MT”, and “CMD”, since any of these should require a fraction of the memory needed during numerical solution. For example, the largest memory requirement is 54m for model FMS 10 using “br CMD”; in this case, one double-precision solution vector will require about 203m. Using MDDs for $\mathcal{S}$ and EV*MxDs, MTMxDs, or CMDs for $\mathbf{R}$, the bottleneck becomes the memory required for solution vectors.

In Table 3, we split generation of $\mathcal{S}$ and $\mathbf{R}$ into separate steps. When building only $\mathcal{S}$, we compare BSTs with MDDs. Perhaps surprisingly, there are a few cases where MDDs are faster than splay trees. For all cases, using batch removal is faster than using single removal. Again, MDDs require orders of magnitude less memory than BSTs. The memory requirements for batch removal are higher than for single removal; one cause for this is the fact that batch removal keeps two MDDs for unexplored states, while single removal keeps only one. Once $\mathcal{S}$ is built, we construct $\mathbf{R}$ by examining each state in $\mathcal{S}$ and determining its outgoing edges based on the model. For BSTs, we examine states of $\mathcal{S}$ in the order in which they were originally discovered; for MDDs instead, we efficiently enumerate the MDD which will visit states in lexicographical order. We believe that this leads to a good, but probably not optimal, order for adding rates to $\mathbf{R}$. For “EV*” and “MT”, we add entries in batch as usual. For CMDs, we can either use our new batch-based implementation, or the original implementation of [19]. The table shows results for both: “bat.” is the batch-based implementation, and

“ $f =$ ” is the original implementation, which merges the CMDs every time the MDD enumeration crosses level f . For all cases, “EV*” is faster than “CMD”, and batch-based CMD is faster than MT. Note that setting the f parameter can be tricky and model-dependent, while a fixed batch size of 1024 gives fairly consistent results. Also note that, except for the Courier model, “EV*” is faster than BST, and for models FMS and Kanban, BST is actually the *slowest* to generate **R**. For many of the models and for EV*s, MTMxDs, and CMDs, the total time for the two-pass algorithm is *lower* than the total time for the one-pass algorithm. As for memory, the trends noted in the “one-pass” table are also seen in the “two-pass” table: BST requires orders of magnitude more memory than the others, with “EV*” requiring the least, followed by “MT”. “—” are cases where BST needed more memory than was available, and did not complete.

In Table 4, we report our preliminary results obtained by incorporating the one-pass algorithms into GreatSPN. There is a clear improvement in CPU and peak memory usage as we progress from “No batches” to “sr” and then to “br”. Note that the CPU usage of “br” for EV* is close to GreatSPN’s BST implementation (WNRG). The peak memory usage of WNRG was not available but we expect it to be similar to the BST implementation used for Table 2.

6 Conclusion

In this paper, we have examined the use of DDs for efficiently storing the state space and Markov chain of a high-level model. We have presented algorithms that improve the performance of DDs when used by traditional explicit generation algorithms. We have also presented small changes to the traditional explicit generation algorithms to take advantages of efficient DD operations.

Our experimental results show that DDs can greatly reduce the peak memory requirements of traditional explicit generation algorithms. There is often the typical time-space tradeoff between a classical BST-based implementation and the DD-based implementations. For the models we tested, BSTs can be nearly a factor of 4 faster than the fastest DD implementation, but at the expense of using on the order of 1000 times as much memory. Models for which traditional explicit implementations exceed the available memory are prime candidates for the DD-based explicit generation. Additionally, for applications where a two-pass generation algorithm is used anyway, we find that DDs are quite competitive with BSTs, sometimes even outperforming them in generation times.

An unexpected result is that EV*MxDs tend to be faster than MTMxDs. It is possible that the cost of the expensive EV*MxD reduction rules is insignificant compared to the cost of accessing more memory (EV*MxDs tend to be more compact than MTMxDs); further analysis of this phenomenon is needed.

Our preliminary tests with GreatSPN using the one-pass algorithm, indicate a clear improvement over previous work in [1]. Once fully integrated we plan on further experiments with generating lumped Markov chains, where the computational cost of determining the next (abstract) state from the high-level model is significantly higher.

Another direction for further work is to study different state exploration orders and develop heuristics for selecting an order that is good in terms of the union operations, and efficient in enumerating an MDD of unexplored states.

Acknowledgment

The authors thank Elvio Amparore and Marco Beccuti for implementing the one-pass algorithms in GreatSPN and running experiments.

References

1. Babar, J., Beccuti, M., Donatelli, S., Miner, A.: GreatSPN enhanced with decision diagram data structures. In: 31st Int. Conf. on Applications and Theory of Petri Nets. pp. 308–317. LNCS 6128 (Jun 2010)
2. Babar, J., Miner, A.: Meddly: Multi-terminal and Edge-valued Decision Diagram Library. In: 7th Int. Conf. on Quantitative Evaluation of Systems (QEST’10). pp. 195–196 (Sep 2010)
3. Baier, C., Haverkort, B., Hermanns, H., Katoen, J.P.: Model checking algorithms for continuous-time Markov chains. *IEEE Trans. Softw. Eng.* 29(6), 524–541 (Jun 2003)
4. Bryant, R.E.: Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.* C-35(8), 677–691 (Aug 1986)
5. Burch, J.R., Clarke, E.M., McMillan, K.L., Dill, D.L., Hwang, L.J.: Symbolic model checking: 10^{20} states and beyond. *Information and Computation* 98(2), 142–170 (Jun 1992)
6. Chiola, G., Franceschinis, G., Gaeta, R., Ribaud, M.: GreatSPN 1.7: Graphical Editor and Analyzer for Timed and Stochastic Petri Nets. *Perf. Eval.* 24(1-2), 47–68 (Nov 1995)
7. Chiola, G.: Compiling techniques for the analysis of stochastic Petri nets. In: Proc. 4th Int. Conf. on Modelling Techniques and Tools for Performance Evaluation. pp. 13–27 (1989)
8. Chiola, G., Dutheil, C., Franceschinis, G., Haddad, S.: Stochastic well-formed colored nets and symmetric modeling applications. *IEEE Trans. Comp.* 42(11), 1343–1360 (Nov 1993)
9. Ciardo, G., Lüttgen, G., Miner, A.S.: Exploiting interleaving semantics in symbolic state-space generation. *Formal Methods in System Design* 31(1), 63–100 (Aug 2007)
10. Ciardo, G., Miner, A.S., Wan, M.: Advanced features in SMART: the Stochastic Model checking Analyzer for Reliability and Timing. *SIGMETRICS Perform. Eval. Rev.* 36(4), 58–63 (Mar 2009)
11. Ciardo, G., Trivedi, K.S.: A decomposition approach for stochastic reward net models. *Perf. Eval.* 18(1), 37–59 (1993)
12. Couvreur, J.M., Thierry-Mieg, Y.: Hierarchical decision diagrams to exploit model structure. In: Proc. 25th Int. Conf. on Formal Techniques for Networked and Distributed Systems (FORTE’05). pp. 443–457 (2005)
13. Delamare, C., Gardan, Y., Moreaux, P.: Performance evaluation with asynchronously decomposable SWN: implementation and case study. In: 10th Int. Workshop on Petri Nets and Performance Models (PNPM’03). pp. 20–29. *IEEE Comp. Soc. Press* (Sep 2003)

14. Derisavi, S.: A symbolic algorithm for optimal Markov chain lumping. In: Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS). pp. 139–154. Springer-Verlag (2007)
15. Hermanns, H., Kwiatkowska, M., Norman, G., Parker, D., Siegle, M.: On the use of MTBDDs for performability analysis and verification of stochastic systems. *Journal of Logic and Algebraic Programming: Special Issue on Probabilistic Techniques for the Design and Analysis of Systems* 56(1-2), 23–67 (2003)
16. Kam, T., Villa, T., Brayton, R., Sangiovanni-Vincentelli, A.: Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic* 4(1-2), 9–62 (1998)
17. Lai, Y.T., Pedram, M., Vrudhula, S.B.K.: Formal verification using edge-valued binary decision diagrams. *IEEE Trans. Comp.* 45, 247–255 (Feb 1996)
18. Miner, A., Parker, D.: Symbolic representations and analysis of large state spaces. In: *Validation of Stochastic Systems*, pp. 296–338. LNCS 2925, Springer-Verlag (2004)
19. Miner, A.S.: Efficient solution of GSPNs using Canonical Matrix Diagrams. In: 9th Int. Workshop on Petri Nets and Performance Models (PNPM'01). pp. 101–110. IEEE Comp. Soc. Press (Sep 2001)
20. Miner, A.S.: Implicit GSPN reachability set generation using decision diagrams. *Perf. Eval.* 56(1-4), 145–165 (Mar 2004)
21. Model Checking Contest at Petri Nets 2014. <http://mcc.lip6.fr/models.php>
22. Molloy, M.K.: Performance analysis using stochastic Petri nets. *IEEE Trans. Comp.* 31(9), 913–917 (Sep 1982)
23. Muppala, J.K., Ciardo, G., Trivedi, K.S.: Modeling using Stochastic Reward Nets. In: Proc. 1st Int. Workshop on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS'93). pp. 367–372. IEEE Comp. Soc. Press, San Diego, CA, USA (Jan 1993)
24. Pastor, E., Roig, O., Cortadella, J., Badia, R.: Petri net analysis using boolean manipulation. In: 15th Int. Conf. on Applications and Theory of Petri Nets. pp. 416–435. Springer-Verlag (Jun 1994)
25. Tilgner, M., Takahashi, Y., Ciardo, G.: SNS 1.0: Synchronized Network Solver. In: 1st International Workshop on Manufacturing and Petri Nets. pp. 215–234. Osaka, Japan (Jun 1996)
26. Wan, M., Ciardo, G., Miner, A.S.: Approximate steady-state analysis of large Markov models based on the structure of their decision diagram encoding. *Perf. Eval.* 68(5), 463–486 (2011)
27. Wimmer, R., Braitling, B., Becker, B., Hahn, E.M., Crouzen, P., Hermanns, H., Dhama, A., Theel, O.: Symbolic calculation of long-run averages for concurrent probabilistic systems. In: 7th Int. Conf. on Quantitative Evaluation of Systems (QEST'10). pp. 27–36 (2010)
28. Woodside, C.M., Li, Y.: Performance Petri net analysis of communications protocol software by delay-equivalent aggregation. In: 4th Int. Workshop on Petri Nets and Performance Models (PNPM'91). pp. 64–73. IEEE Comp. Soc. Press, Melbourne, Australia (Dec 1991)
29. Xie, A., Beerel, P.A.: Efficient state classification of finite state Markov chains. *IEEE Trans. Computer-Aided Design* 17(12), 1334–1339 (1998)
30. Yoneda, T., Hatori, H., Takahara, A., Minato, S.: BDDs vs. zero-suppressed BDDs: for CTL symbolic model checking of Petri nets. In: FMCAD'96. pp. 435–449. Springer-Verlag (1996)